Trukio

Product Definition & Technical Blueprint

Living Document — Last Updated: March 2026

Status: In Planning | Stage: Pre-Build

---

1. Product Overview

A mobile-first logistics operations platform for small freight operators in India. Replaces WhatsApp + notebook-based workflows with a structured, AI-assisted operations cockpit — while keeping data ownership with the operator.

Positioning

"Operations manager in your pocket — for transporters who run on WhatsApp and phone calls today."

Target Market

- Small freight operators in Mumbai and similar Indian metros
- Operators managing 2–50 drivers, receiving orders via WhatsApp/phone
- Businesses currently tracking deliveries via calls and maintaining invoices in physical booklets
- First target validation: operator in Mumbai (founder's brother)

Core Problem Being Solved

- Orders tracked in WhatsApp + memory — high risk of missed or wrong deliveries
- Driver location tracked only via phone calls — no visibility
- Manual invoice booklets — no payment tracking, no digital records
- Language barrier — operators more comfortable in Hindi/Marathi than English
- Data privacy concern — operators wary of their business data being stored on third-party clouds

---

2. Competitive Landscape

Existing players fall into two categories — neither serves this product's niche.

Player	Type	Gap
Vahak, BlackBuck, TrucksUp	Load board / marketplace	Helps find trucks — not manage your own operation

Player	Type	Gap
WheelsEye, Porter	Booking platform	Intra-city / booking focused, not ops management
Bharat Software, NYGGS	Enterprise TMS	Desktop-first, complex, not mobile-native or vernacular
This Product	Ops cockpit	Fills the gap — manage after order is received

Key differentiators: vernacular-first (Hindi/Marathi/English), mobile-native, local-first data ownership, designed for operators who already have their own customers and trucks.

---

3. Product Architecture

3.1 App Structure

Single app, role-based interfaces. One Play Store / App Store listing. Owner logs in → full ops dashboard. Driver logs in → jobs and location sharing only.

3.2 Owner Interface — Features

Order Management

- Log new order via chat (voice or text, Hindi/Marathi/English/mixed)
- Manual form entry as alternative to chat
- System extracts: pickup location, drop location, material type, quantity, date
- View all orders — active, pending, completed
- Edit order details manually
- Assign order to driver
- Cancel an order

Driver Management

- Add driver (name, phone number)
- Invite driver via WhatsApp link
- Set driver active/inactive
- View which driver is on which job

Live Tracking

- All active deliveries on single map
- Animated truck icon moving along route
- Route line drawn from current position to destination
- Each driver pin shows job info on tap

- Last updated timestamp visible

Invoicing

- Auto-generated when job marked delivered
- Pulls from order data — customer name, material, pickup, drop, date
- Owner adds price before generating
- Multi-page support, A5 default, letter size option
- Download as PDF, share via WhatsApp
- Predefined templates (V1), custom template designer (V2)
- Invoice numbering chosen at onboarding — auto-generate or custom format
- Small 'Powered by Trukio' branding in corner
- One-time payment to remove watermark

Dashboard

- Today's job count
- Pending payments count
- Active drivers on map
- Recent order list

3.3 Driver Interface — Features

- See assigned job — pickup address, drop address, material
- One-tap status: Accepted → Picked Up → On the Way → Delivered
- View job history
- Location shared only during active job
- Location sharing stops automatically on delivery confirmation
- Background location with visible notification — 'Your location is being shared for this delivery'

3.4 Data & Sync

- All data stored locally on owner's device (WatermelonDB)
- Auto backup to owner's Google Drive when on WiFi
- Manual 'Backup Now' option always available
- Toggle in settings: 'Also sync on mobile data'
- Restore from Drive on new device
- No data stored on product servers — privacy promise

---

4. Functional Requirements

#	Requirement	Interface	Version
FR-01	Log order via voice in Hindi/Marathi/English	Owner	V1
FR-02	Log order via text chat	Owner	V1
FR-03	Log order via manual form	Owner	V1
FR-04	Assign order to driver	Owner	V1
FR-05	View all orders by status	Owner	V1
FR-06	Live map of all active drivers	Owner	V1
FR-07	Animated truck icon + route line on map	Owner	V1
FR-08	Auto-generate PDF invoice on delivery	Owner	V1
FR-09	Share invoice via WhatsApp	Owner	V1
FR-10	Invite driver via WhatsApp link	Owner	V1
FR-11	Phone OTP login	Both	V1
FR-12	Google Sign-In option	Both	V1
FR-13	One-tap delivery status updates	Driver	V1
FR-14	Background location sharing during job	Driver	V1
FR-15	Backup to Google Drive (WiFi auto, data manual)	Owner	V1
FR-16	Route progress push notifications	Owner	V2
FR-17	Custom invoice template designer	Owner	V2
FR-18	Continuous background Drive sync	Owner	V2

5. Non-Functional Requirements

Category	Requirement
Platform	Android first, iOS second
Connectivity	Core management works fully offline (orders, drivers, invoices, history)
Connectivity	Live map and chat/LLM require internet — acceptable
Language	Supports Hindi, Marathi, English and mixed-language input
Performance	Invoice PDF generates in under 3 seconds
Performance	Driver location updates every 15 seconds during active job
Usability	Designed for non-tech-savvy users — icon-heavy, minimal text
Privacy	No business data stored on product servers
Scale	Supports up to 50 drivers per owner account (Fleet tier)

---

6. Technology Stack

Decision	Choice	Reason
Mobile Framework	React Native + Expo	TypeScript fit, largest ecosystem, strongest AI coding assistance, Expo removes iOS build friction
Backend	Supabase (PostgreSQL)	Relational data model, realtime built-in, row-level security for multi-tenant, open source
Local Database	WatermelonDB	Purpose-built offline-first for React Native, handles sync patterns
Map Display	react-native-maps (Google Maps forced on both platforms)	Best Indian street data, consistent experience across Android and iOS
GPS / Location	expo-location	Abstracts over Android and iOS location APIs cleanly
LLM for Chat	Gemini Flash	Best Hindi/Marathi support, cheapest, lower latency from Indian servers
PDF Generation	react-native-html-to-pdf	Local on device, free, offline-capable, full HTML/CSS control
Authentication	Phone OTP (primary) + Google Sign-In (secondary)	Phone number is how operators identify in business — lowest friction
Drive Sync	Google Drive API	Owner's own Drive — aligns with data ownership promise
Payments	Stripe	Handles subscriptions, promo codes, one-time payments (watermark removal)
Realtime Location	Supabase Realtime	Driver coordinates pushed to owner map via Supabase channels

7. Pricing Model

Plan	Price/month	Driver Seats	Target	Custom Materials
Starter	₹299	5 drivers	Just starting, 1–2 trucks	5
Growth	₹599	20 drivers	Established small operator	30
Fleet	₹999	50 drivers	Larger operation	Unlimited

Pricing Notes

- No permanent free tier — use promo codes for early adopters, referrals, and direct outreach
- Drivers are free seats — owner pays, drivers never charged
- One-time payment available to remove 'Powered by Trukio' watermark from invoices
- Stripe handles all billing, subscriptions, and promo codes
- Pricing to be validated after 20–30 signups — adjust if users cluster at one tier

8. Invoice Specification

Page Size

- Default: A5
- Option: Letter size (selectable)
- Multi-page supported

Standard Fields (Indian GST Compliance)

- Business header — company name, address, phone, GST number, optional logo
- Invoice details — invoice number, date, vehicle number
- Consignment — From, To, material type, quantity/weight, number of units
- Charges — freight, loading/unloading, other charges, total
- Payment terms — due date, payment mode, optional bank details
- Footer — terms, signature line, 'Powered by Trukio' branding

Invoice Numbering

- Set during onboarding — owner chooses auto-generate or custom format
- Auto: INV-2025-001 incrementing
- Custom: owner sets prefix (e.g. RAM/2025/), app auto-increments
- GST compliance — no gaps or resets in numbering
- Changeable in settings after onboarding

9. Database Schema

9.1 Design Principles

- Supabase PostgreSQL is source of truth — WatermelonDB is offline cache only
- Row-level security on every table enforced at database level, not application level
- Denormalize name fields on orders and invoices — protects historical accuracy if records renamed
- audit_log is append-only — no updates or deletes ever
- Single device per owner assumed for V1 — no conflict resolution needed

9.2 Table Overview

#	Table	Purpose
1	owners	Business accounts — one per freight operator
2	drivers	Driver profiles — standalone, no hard owner attachment
3	team_memberships	Owner-driver relationships — permanent or temporary
4	material_catalogue	Owner item list with names and units
5	orders	Incoming customer requests
6	jobs	Assigned delivery execution — one per order
7	job_locations	Last known driver position — one row per job updated in place
8	invoices	Generated billing records
9	invoice_templates	HTML templates for PDF generation
10	subscriptions	Plan and billing history
11	notifications	In-app and push notification state
12	audit_log	Immutable change history for disputes

9.3 Key Design Decisions

Driver Flexibility

A driver has no hard attachment to any owner. team_memberships manages all relationships.

- Permanent team driver: membership_type = permanent, end_date = null
- One-day external hire: membership_type = temporary, end_date = today
- Driver leaving: is_active = false on membership record
- Driver joining new owner: new membership record, full history preserved
- Owner bench: list of past temporary drivers reactivatable in one tap

Material Catalogue

Owner defines their own catalogue in profile settings — items and units they actually use. Voice and chat orders match spoken material to catalogue via Gemini. Unrecognised material prompts owner to add it. Quantity stored as structured value + unit, not free text.

Driver Access

- All drivers must install the app — permanent and temporary
- Zero-install web option rejected for V1 — mobile browser background location breaks core tracking feature
- Customer-facing public tracking link: V2 — share_token column already on jobs table, ready to activate

Invoice Numbering

- Chosen at onboarding: auto-generate (INV-2025-001) or custom prefix (RAM/2025/)
- UNIQUE(owner_id, invoice_number) — GST compliance, no gaps or resets

9.4 RLS Summary

Table	Owner Access	Driver Access
owners	Own row only	None
drivers	All drivers in active memberships	Own row only
team_memberships	All memberships where owner_id = uid	Own memberships only
material_catalogue	Own catalogue only	Read-only via active job
orders	Own orders only	Read via assigned job
jobs	Own jobs only	Own jobs — status update only
job_locations	Read where owner_id = uid	Write own location only
invoices	Own invoices only	None
notifications	Own notifications	Own notifications
audit_log	Own logs — read only	None

---

10. Open Decisions & Next Steps

10.1 Decisions Remaining

#	Decision	Status
D-01	App name / brand identity	Open
D-02	Onboarding flow design	Open
D-03	Chat agent tool calling spec	Open
D-04	Project folder structure and code conventions	Next
D-05	Module build order	Next
D-06	First Claude Code build prompt	Next

10.2 Immediate Next Steps

- Step 3 — Project folder structure and code conventions
- Step 4 — Module build order
- Step 5 — Claude Code prompts, one per module
- Name the app

---

11. System Architecture

11.1 Architecture Overview

Supabase is the source of truth. WatermelonDB is the offline cache on device. All business data lives in Supabase PostgreSQL with row-level security — each owner can only ever access their own data, enforced at the database level not the application level.

11.2 Privacy Promise

Your data is stored securely and privately. We never access, sell, or mine it. You can export or delete everything at any time.

11.3 The Five Core Flows

Flow 1 — Authentication

- Owner enters phone number → Supabase sends OTP → session token stored locally → role check loads correct interface
- Driver onboarding: owner sends WhatsApp invite link → driver taps link → phone OTP → auto-linked to owner's team

Flow 2 — Order Creation (Three Paths, One Outcome)

- Voice: owner speaks → Gemini Flash transcribes + extracts order via tool calling → owner confirms → saved to WatermelonDB → synced to Supabase
- Chat text: owner types in any language → Gemini Flash extracts → same confirm/save flow
- Manual form: owner fills fields directly → no LLM involved → saved directly
- All three paths produce identical order objects — same data structure regardless of input method
- If offline: order saves locally, banner shows Not synced — driver cannot see this job yet. Driver notified only after sync

Flow 3 — Live Driver Tracking

- Driver accepts job → expo-location starts background tracking → coordinates pushed to Supabase Realtime channel every 15 seconds
- Owner map subscribes to all active job channels → truck icon animates to new position → route line redraws to destination
- Storage: last known location only — one row per job updated in place, not full trip history
- Timestamps stored: job pickup time and delivery time — estimated vs actual arrival calculated from these
- Job completed: location sharing stops, channel closes, truck removed from map, invoice generation triggered

Flow 4 — Invoice Generation

- Driver marks Delivered → app pulls order data → populates HTML template → react-native-html-to-pdf converts on device
- PDF saved to device → owner notified → options: Download / Share via WhatsApp / Print
- PDF included in next Google Drive backup

Flow 5 — Google Drive Sync

- WiFi connected + last backup over 24 hours: auto export WatermelonDB snapshot as JSON + invoice PDFs → zipped → uploaded to owner Drive
- Keep last 5 backups, delete older ones automatically
- Manual Backup Now always available regardless of connection type
- Restore on new device: owner logs in → picks Drive backup → JSON imported → PDFs downloaded

11.4 Component Responsibilities

Component	Role	Notes
Supabase PostgreSQL	Source of truth for all business data	Row-level security per owner
Supabase Realtime	Live driver location channels	One channel per active job, closed on delivery

Component	Role	Notes
Supabase Auth	Phone OTP + Google Sign-In	Session tokens stored locally on device
WatermelonDB	Offline cache on device	Queues writes when offline, flushes when online
Gemini Flash	Chat and voice order extraction	Tool calling returns structured order object
expo-location	GPS coordinates from driver device	Background mode during active job only
react-native-maps	Map display — Google Maps forced on both platforms	Animates truck icon, draws route line
react-native-html-to-pdf	Invoice PDF generation	Runs locally on device, no internet needed
Google Drive API	Long-term backup	Owner's own Drive — auto on WiFi, manual on mobile data
Stripe	Billing and subscriptions	Handles tiers, promo codes, watermark removal

11.5 Scalability Guarantees

- Row-level security enforced at database level — one Supabase project serves all operators safely with zero data leakage risk
- Realtime channels only exist during active jobs — no persistent idle connections draining quota
- WatermelonDB offline cache means app is fully functional without internet — no dependency on server uptime for core operations
- Google Drive backup is independent safety net — survives Supabase outages

11.6 Stripe Webhook Architecture

Webhooks are the source of truth for all subscription state. The database never updates subscription status from a button click — only from confirmed Stripe webhook events.

The Right Pattern

- User clicks Subscribe / Cancel / Upgrade in app
- App calls Stripe API — initiates the action only
- App shows processing state — never assumes success
- Stripe processes the request
- Stripe sends webhook event to Supabase Edge Function endpoint
- Edge Function verifies Stripe signature, updates database
- Database now reflects reality — not the button click

If Stripe Is Down

- App calls Stripe API → request fails → app shows error → user tries again
- Database unchanged — no mismatch, no ghost subscriptions
- Stripe retries failed webhook deliveries automatically for up to 3 days on recovery

Webhook Events Handled

Event	Action
customer.subscription.created	Activate plan, update owners.plan_status
customer.subscription.updated	Update tier, update owners.plan_tier
customer.subscription.deleted	Deactivate plan after grace period
invoice.payment_failed	Mark payment_failed, show banner to owner
invoice.payment_succeeded	Confirm active, clear any warning banners

Payment Failure Grace Period

- Payment fails → mark owner as payment_failed → show banner: Payment failed — update card within 7 days
- Stripe retries automatically — 3 attempts over 7 days
- Retries succeed → webhook fires → banner disappears → access restored
- All retries fail → webhook fires → plan deactivated
- Purpose: prevents losing good customers over temporary card issues — common with Indian cards and international processors

Edge Function Cost

- Stripe webhook handler runs as Supabase Edge Function
- Cost: $2 per million invocations — effectively free at any realistic scale
- Spend cap enabled by default on Pro plan — protects against surprise bills

11.7 Supabase Cost Roadmap

Stage	Approx Users	Supabase Cost
Building and beta	0–50 operators	$0 (Free plan)
First customers	50–500 operators	$25/month (Pro)
Growing	500–5,000 operators	$25–50/month (Pro)
Scaling	5,000–50,000 operators	$50–150/month (Pro)

- Free plan warning: projects pause after 1 week of inactivity — never run live product on free plan
- Move to Pro ($25/month) the moment first paying customer signs up
- Spend cap: enable on Pro plan — prevents surprise bills even if a bug causes runaway queries
- Cost stays low because this app has no media or heavy files — mostly text data, gentle bandwidth curve

12. Project Folder Structure

Feature-first organisation. Shared things in shared folders. Strict layer separation — services are the only place external APIs are touched.

Top-level structure (src/)

- app/ — Expo Router file-based navigation. Route groups: (owner)/, (driver)/, (auth)/
- features/ — Feature modules. Each contains components/, hooks/, types.ts. Modules: orders, drivers, jobs, tracking, invoices (includes templates/), chat, auth
- db/ — WatermelonDB layer. schema.ts, migrations/, models/ (Order, Job, Driver, Invoice, Notification)
- services/ — External service clients (only place Supabase/APIs are touched). Subfolders: supabase/, gemini/, maps/, drive/, stripe/, pdf/
- sync/ — Offline sync layer. SyncManager.ts, queue.ts, strategies/ (per-table sync strategies)
- shared/ — Truly shared across features. components/ui/ (Button, Input, Card, Badge, Modal, LoadingSpinner), hooks/ (useNetworkStatus, usePermissions, useAppState), utils/, constants/, types/
- store/ — Zustand global state. authStore.ts (user, role, session), syncStore.ts (sync status, pending count), uiStore.ts (toasts, modals)

Naming conventions

- Screens: lowercase | Components: PascalCase | Hooks: camelCase with use prefix | Services: camelCase | Models: PascalCase

13. Module Build Order

Build order is driven by three constraints: dependency (some modules block others), validation speed (get something in the operator's hands fast), and risk (hardest pieces stay off the critical path). Target: full flow — create order → assign → driver sees job → owner tracks live.

Phase 1 — Skeleton (Days 1–2)

Goal: app launches, navigates, nothing crashes

- Project setup — Expo + React Native, folder structure, Supabase client, WatermelonDB schema, Zustand stores, navigation shell (owner/driver route groups)
- Auth — Phone OTP flow, Google Sign-In, role detection, session persistence, driver invite link handling

Phase 2 — Core Loop (Days 3–7)

Goal: owner creates order, assigns driver, driver sees it

- WatermelonDB models + sync foundation — Order, Job, Driver, Invoice models; basic SyncManager; offline banner
- Driver management — invite flow, team membership, driver list UI
- Order creation (manual form only) — nail the data model first before adding voice/chat complexity
- Job assignment — assign order to driver, driver accepts/rejects, status transitions

Phase 3 — Live & Trackable (Days 8–11)

Goal: owner sees truck moving on map

- Live tracking — expo-location background, Supabase Realtime channel, owner map with animated truck icon
- Job status flow — picked up → in transit → delivered, timestamps, driver UI

Phase 4 — Money (Days 12–15)

Goal: usable end-to-end for real work

- Invoicing — HTML template, react-native-html-to-pdf, WhatsApp share
- Notifications — in-app only for V1, job state change alerts

Phase 5 — After Validation

- Stripe subscriptions — only after confirming people want to pay
- Voice + chat order input (Gemini Flash) — after manual form is battle-tested
- Google Drive backup
- Material catalogue (all tiers, limits enforced: Starter 5, Growth 30, Fleet unlimited)

14. Environment Variables

All external service credentials are stored in environment variables. No hardcoded values anywhere in the codebase. The APP_ENV variable controls behaviour across environments (logging verbosity, Sentry sampling, mock data). Handover is a .env swap only — no code changes required.

Required variables

- EXPO_PUBLIC_SUPABASE_URL — Supabase project URL
- EXPO_PUBLIC_SUPABASE_ANON_KEY — Supabase anon/public key
- SUPABASE_SERVICE_ROLE_KEY — Server-side only. Never expose to client.
- EXPO_PUBLIC_GOOGLE_MAPS_API_KEY — Google Maps for react-native-maps
- GEMINI_API_KEY — Google Gemini Flash for voice/chat order entry
- EXPO_PUBLIC_STRIPE_PUBLISHABLE_KEY — Stripe client-side key
- STRIPE_SECRET_KEY — Supabase Edge Function only. Never in app bundle.
- STRIPE_WEBHOOK_SECRET — Supabase Edge Function only
- APP_ENV — development | staging | production. Controls logging, Sentry sampling rate, and mock data flags.

Monitoring & error tracking

- SENTRY_DSN — Add when first real user onboards. Free tier covers 5,000 errors/month.

Logger pattern

A logger utility (src/shared/utils/logger.ts) wraps all console calls. In development, everything logs to console. In production, errors route to Sentry and all other logs are suppressed. Use logger.x everywhere — never call console.x directly in feature code.

15. Change Log

Date	Change	Section
March 2026	Stripe webhook architecture and Supabase cost roadmap added	Section 10
March 2026	Database schema added — 12 tables, RLS policies, driver flexibility, material catalogue	Section 9
March 2026	System architecture added — five flows, component map, scalability decisions	Section 10
March 2026	Initial document created — requirements, stack, pricing, invoice spec all locked	All

16. Handover Note

Purpose

This document was created during the development phase of Trukio by the original developer. When the codebase is handed over to the operating company, this section should be reviewed in full and all items marked for re-verification should be confirmed against live service pricing, policies, and account configurations at the time of handover.

Handover Checklist

The following actions are required at handover time:

- Create a new Supabase project under the company's account and upgrade to Pro ($25/month). Do not use the development project — it contains no real data and was run on the Free tier which pauses after 1 week of inactivity.
- Create a new Expo account under the company's email. Transfer or rebuild the project under that account. The development Expo account belongs to the original developer.
- Update all environment variables in the new production .env file: Supabase URL and keys, Gemini API key, Google Maps API key, Stripe keys, and APP_ENV=production.
- Register Apple Developer Account ($99/year) and Google Play Developer Account ($25 one-time) under the company. These are required to publish to app stores and cannot be transferred from a personal account.
- Register domain (trukio.in recommended) and set up business email (e.g. hello@trukio.in) before go-live.
- Create Stripe account, configure subscription products matching the three Trukio pricing tiers (Starter ₹299, Growth ₹599, Fleet ₹999), and update Stripe keys in environment variables.
- Run all Supabase migrations on the new production project and verify RLS policies are active on every table before onboarding the first operator.

Infrastructure Cost Summary

Expected monthly running costs once live. Fixed costs are predictable; variable costs scale with operator count and transaction volume.

Service	Early Stage (0–50 operators)	Growth (170+ operators)	Notes
Supabase	$25/month	$25/month	Pro plan required from day one of live ops. Never use Free tier for production.
Expo EAS	$0	$19/month (Starter)	⚠️ REVERIFY: Paid plan required once revenue exceeds $1,500 USD/month. Confirm fair use policy at handover.
Sentry	$0	$0 (Free tier: 5K errors/month)	⚠️ REVERIFY: Confirm free tier limits at handover.
Google Maps API	~$0–5/month	~$10–20/month	⚠️ REVERIFY: Google Maps pricing changes

			frequently. Check current rates and set billing alerts on the GCP console.
Gemini Flash	~$1–3/month	~$5–10/month	⚠️ REVERIFY: Gemini pricing and free tier limits change. Check current rates at handover.
Stripe	~$8/month (on 20 subs)	~$65/month (on 170 subs)	2.9% + 30¢ per transaction. No monthly fee. Scales directly with revenue.
Apple Developer	$99/year (~$8/month)	$99/year (~$8/month)	Required to publish on iOS App Store. Must be registered under the company, not an individual.
Domain + Email	~₹1,500/year (~$2/month)	~₹1,500/year (~$2/month)	⚠️ REVERIFY: Check trukio.in availability at handover. Price may vary by registrar.
Estimated Total	**~$43/month (~₹3,600)**	**~$120/month (~₹10,000)**	**At 170 operators, monthly revenue exceeds ₹50,000. Infrastructure is ~20% of revenue.**

⚠️ Items marked REVERIFY must be confirmed against live pricing and policies at the time of handover. All costs in this document were estimated during the development phase and are subject to change.